MORPHO

Overview and Issue

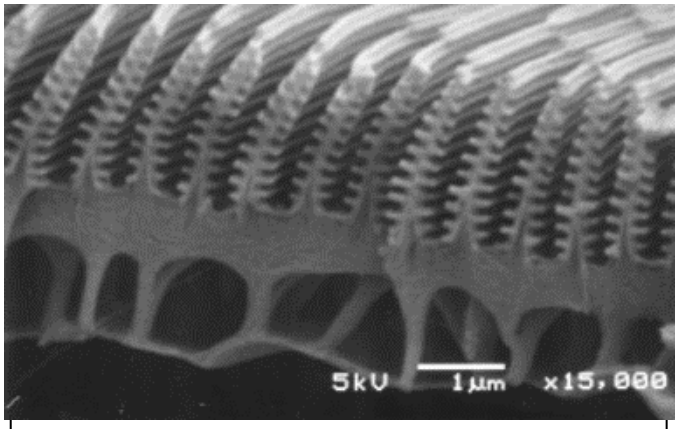
Our project idea is called Morpho, and it consists of a sustainable wallpaper that can morph into any image the consumer desires, uploaded through an app on their device. It provides a more convenient, flexible and adept way for individuals to decorate their home.

Our product also aims to replace paint, which has the potential to harm the environment as well as consumers. It contains chemicals called volatile organic compounds (VOCs), which the paint releases into the air as gases. Examples of VOCs include toluene, xylene and benzene, which will continue to be released throughout the paint's entire lifetime. [1][2]

The USA's Environmental Protection Agency classified paint as one of its top five most hazardous substances. This is because VOCs vaporise easily and react with oxygen and form harmful ozone when exposed to sunlight, which contributes to the greenhouse effect and is a cause for global warming. [3]

Furthermore, studies have shown that exposure to VOCs can lead to several negative health effects. Short-term effects include irritation of the eyes, nose or throat, headaches, nausea, trouble breathing and it can also trigger conditions such as asthma. In extreme cases, long-term exposure to VOCs can also lead to more severe damage to the nervous system, the liver or the kidneys. [1][2]

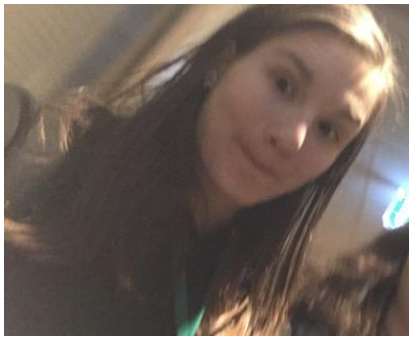
We've also incorporated the ideas of biomimicry into our project by drawing inspiration from the Menelaus blue morpho butterfly, an animal that we learnt about when attending a physics society meeting at school. The Morpho butterfly is one of the largest butterflies in the world, with a wingspan from 5 up to 8 inches. They are known for their vivid, iridescent blue colouring, which is actually not a result of pigmentation, but instead from microscopic, tree-like structures on their wings. These structures serve to refract light, causing constructive interference, which leads to the light waves having a wavelength corresponding to the blue section of the visible spectrum (see the Physics of Light section on page 4 for more information on how this process occurs). Furthermore, if these structures are tilted, the process of refraction and reflection produces light of different colour. For our final product, we decided to design the structures with only one layer, since this was more suited towards our purposes. Utilising this concept, our wallpaper is made up of an array of these microscopic structures that, through the control of a motor, tilt to produce different images.



Tree-like structures on the wings of Morpho

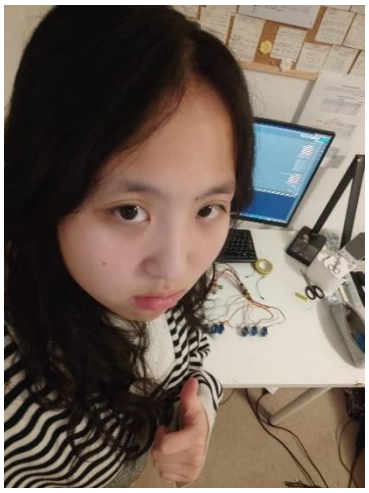
Our project also involved the research and employment of many other different aspects of technology, such as the concept of RGB pixels inspired by those used in Apple displays, app development using Kivy, Bluetooth for linking the app to the product, building circuits with Arduinos and servo motors for the prototype and using Fusion 360 to design and 3D print our prototype. There was also a significant amount of physics and maths involved in ensuring the success of our product.

About Us



Silvia Muresan

My role in this project was mainly theoretical. I researched thin film interference and coded the app and python scripts, bridging the gap between an image and the output for the servo motors in the prototype. I found the most fitting materials for the project without compromising sustainability, and sought to perfect our pixels, inspired by modern device screens. I am interested in software engineering, yet I enjoyed the strong links between wave optics and mathematics. I was surprised to discover how tightly knit computer science and physics are, and just how useful computer programs can be when performing calculations.



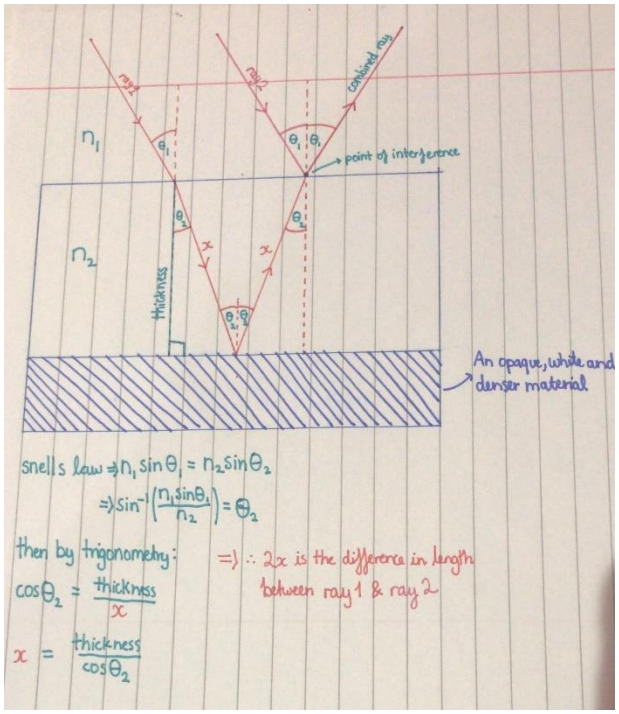
Zhixuan (Sharon) Wang

My role in this project was mainly focused on the practical aspects. I helped turn our theoretical ideas into physical products, which involved finding ways to make everything doable. This required intense scrutinisation of all the components and leaning about new technologies such as Bluetooth. I was also predominantly responsible for the building of our prototype, which was the cause for much blood, sweat and tears in the form of broken motors, tangled wires and an immense amount of both frustration and patience. However, overall, I found this project very rewarding, having gleaned many practical and collaboration skills from it. I was inspired to continue physical computing, as well as having developed a newfound interest in biomimicry and nanotechnology.

Contents

Overview and Issue 1
About Us 2
Physics of Light 3
Pixel Layout 4
The App 4
Thin Film Calculations 6
Materials 7
Design and Innovation 8
The Prototype 11
Next Steps and Marketing 13
Skills and Lessons Learnt 13
External Support 14
Bibliography 14

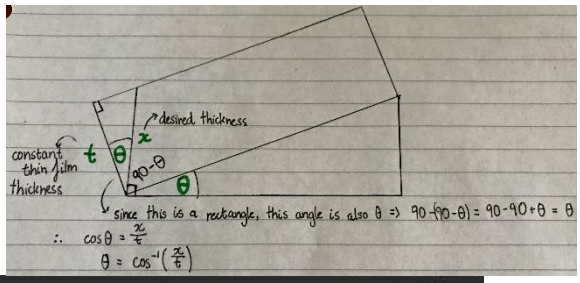
Physics of Light



Our design uses the concept of thin film interference, the interference of light reflected from the top and bottom of thin film, a film with the thickness a few times smaller than the wavelength of light. As light enters the film, some of it will be reflected off the surface and some will enter the film, refract, and then be reflected off the bottom of the film. However, the light which has been reflected off the bottom of the film has travelled a greater distance than the one reflected off the top, so when they interfere, the waves may no longer be in phase with one another, resulting in either constructive or destructive interference. Two waves that are in phase are at exactly the same point in their wave cycle. Since the waves started off at the same point, the difference in their wavelength is equal to the extra distance travelled by the second wave through the thin film. Therefore, if this is equal to a multiple of the wavelength of light we are after, the two waves will remain in phase and constructive interference will occur, meaning the amplitudes of the two waves are added, allowing us to see that wavelength much brighter. Destructive interference is the opposite and occurs when the waves are not in phase with one another and instead their wavelengths are exactly half a wavelength apart, meaning that the crest of a wave meets the trough of another. This happens when the path of the two waves differs by half an odd multiple of their wavelength (half an even multiple gives us a full wavelength, resulting in constructive interference). When the amplitudes of these two waves are added together, the overall amplitude is 0, and the two waves cancel each other out, resulting in less or no light of that wavelength. [4] However, if one of the rays gets pi shifted and the other does not, the requirements for destructive and constructive interference swap, as the wave has been flipped, meaning that the peak and the trough of the wave have swapped, affecting the overall amplitude when the two waves interfere. Pi shift occurs when a wave reflects off a denser medium than the medium it was previously travelling in. [5] Therefore, when our first ray reflects off our thin film, it will undergo pi shift. To make our calculations easier, we want the second ray to pi shift as well, meaning the material below our thin film must be denser than the thin film itself (See what materials we used in the Materials section on page 8).

Our Original Idea

Originally, we planned on using this idea and varying the thickness of the thin film by titling it. To calculate the angle of tilt to achieve the desired thickness, we drew this diagram. However, we quickly realised that not all colours are on the visible light spectrum, and




```
import math
wavelength = 400
constant_thickness = 135 #perfect for purple
rough_refractive_index = 1.4661
desired_thickness = (wavelength/rough_refractive_index)/2
angle_of_tilt = math.degrees(math.acos(constant_thickness/desired_thickness))
print(angle_of_tilt)
```

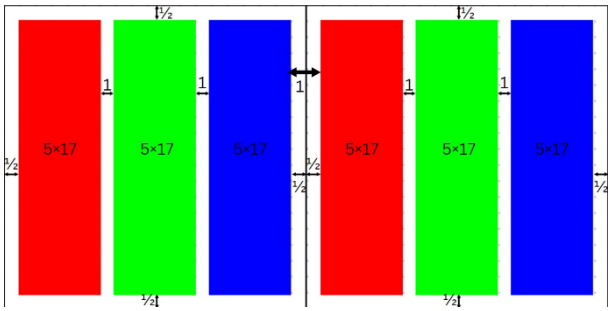
Here, we are importing a library so we can use inverse cosine in our calculations and then establishing the wavelength we want to find, as well as the constant thickness of the thin film and its refractive index.

To calculate our desired thickness, we use the equation $\frac{\lambda}{n_2}$, where λ is the wavelength and n_2 is the refractive index of our thin film. Next, we use the equation from the diagram and convert the final angle of tilt to degrees as the math library works in radians.

instead many are made up of multiple colours. For example magenta is made up of red and blue, and it would therefore be impossible to recreate a picture on a wall with only the palate of colours on the visible light spectrum. Our original code for converting from a wavelength to a desired angle of tilt is shown on the previous page.

Pixel Layout

Therefore, inspired by the subpixel layout of modern device screens, we decided that each pixel would have multiple subpixels responsible for constructively interfering red, green or blue light, and that after extracting an RGB value from the image to be displayed on the wall, we would use this ratio to determine how many of the red, green or blue subpixels would be tilted into their "on" position and able to constructively interfere light. The rest would be tilted 90˚ into an "off" position, the sides of the thin film coated with a black material to absorb the light. (See what material we used in the materials section) Since RGB values are out of 255, we would need 255 subpixels for each of the 3 colours, resulting in 765 subpixel per pixel. This is far too much, considering that thousands of these pixels make up an image and each pixel will measure 1mm by 1mm to optimise our image resolution. Therefore, we decided to divide each RGB value by 3, giving us 85 subpixels per colour and 255 per pixel. With a one subpixel gap between each of the RGB colours, it makes a perfect square ((5+1+5+1+5)×17), which is more appealing to the eye and prevents the image from appearing squashed on the wall. This gap also provides interpixel spacing, common in device screens and essential for increased clarity, resolution, and display quality. Between each pixel, there is also a subpixel-sized gap as seen in the diagram.



The App

We decided to use Kivy to make our app as the majority of the rest of our code was in phyton so it would therefore be easier to upload and access the image to extract its RGB values. The user can also input the width and height of their wall so we can scale their image appropriately. [6] Our editor.kv file:

```
<MyGrid>:
  BoxLayout:
    orientation: 'vertical'
    padding: 100
    spacing: 5
    BoxLayout:
      size_hint_y: None
      height: 30
      Button:
        background_normal: ''
        background_color: .3, .3, 1, .85
        text: 'Load Image'
        on_release: root.show_load()
    BoxLayout:
      size_hint_y: None
      height: 60
      TextInput:
        id: wing_width_input
        hint_text: 'Enter wing width'
        multiline: False

      TextInput:
        id: wing_height_input
        hint_text: 'Enter wing height'
        multiline: False
    BoxLayout:
      size_hint_y: None
      height: 30
      Button:
        background_normal: ''
        background_color: .3, .3, 1, .85
        text: 'Submit'
        on_release: root.get_wing_dimensions()
    BoxLayout:
      Image:
        id: uploaded_image
        source: ''

<LoadDialog>:
  BoxLayout:
    size: root.size
    pos: root.pos
    orientation: "vertical"
    FileChooserListView:
      id: filechooser

    BoxLayout:
      size_hint_y: None
      height: 30
      Button:
        text: "Cancel"
        on_release: root.cancel()

      Button:
        text: "Load"
        on_release: root.load(filechooser.path, filechooser.selection)
```

Here we are creating a new grid called MyGrid and then creating a main BoxLayout to hold the separate BoxLayouts for each of our components.

The first BoxLayout holds the Load Image button, which has been set to blue, once the button has been pressed, our main python file is called and the show_load function is executed.

In the second BoxLayout, two text inputs are created, and each one is given an id so that the values can be accessed from our main python code. A hint_text is added so the user knows where to input the height and where to input the width.

In the third BoxLayout we are creating a blue button called submit which will execute the get_wing_dimensions function when pressed.

In the fourth BoxLayout we are creating an image viewer which will display the selected image. It is originally set to empty before the file is selected.

The LoadDialog is shown when show_load is run. The size and position have been set to be the same as the rest of the app so that they take up the full screen. A filechooser is then displayed to allow the user to select the appropriate image file.

This final BoxLayout is displayed at the bottom of the filechooser and contains two buttons. The cancel button executes the cancel function from the main python file, and the load button executes the load function from the main file, taking in two variables, the path of the chosen file and a list of selected files if there are any.

Main Python Script

```
import os  # a library used in file operations and directory handling
import random  # a library used to reorder lists randomly
from PIL import Image  # a library which allows you to work with images
from kivy.app import App  # imports the App class from the kivy library, which is the base for kivy
applications
from kivy.uix.floatlayout import FloatLayout  # imports a layout manager that arranges widgets
from kivy.factory import Factory  # imports a class used for registering custom widget classes
from kivy.properties import ObjectProperty  # imports a class which allows you to create properties for
widgets
from kivy.uix.popup import Popup  # imports the popup class to allow functions relating to the popup to
be used

class LoadDialog(FloatLayout):  # creates a new class called LoadDialog (this is the LoadDialog from
the editor.kv file)
    load = ObjectProperty(None)  # two object properties which represent the load and cancel actions
    cancel = ObjectProperty(None)
    wing width input = ObjectProperty(None)  # sets up width of the wing as an empty object
    wing_height_input = ObjectProperty(None)  # sets up height of the wing as an empty object

class MyGrid(FloatLayout):  # defines the main application layout
    loadfile = ObjectProperty(None)  # an object property which represents the load file action
    _popup = None  # a private variable to store the popup widget

    def dismiss_popup(self):
        self._popup.dismiss()  # creates a function which dismisses the popup

    def show_load(self):  # function executed when Load Image from the editor.kv file is pressed
        content = LoadDialog(load=self.load, cancel=self.dismiss_popup)  # sets the content of the
popup as LoadDialog
        # when the load button is pressed, the load function is executed
        # when the cancel button is pressed, the popup is dismissed.
        self._popup = Popup(title="Load image", content=content, size_hint=(0.9, 0.9))
        self._popup.open()  # displays the LoadDialog

    def load(self, path, filename):  # defines the load function
        if filename:  # checks that there is a selected file
            image_path = filename[0]  # selects the first file and saves its full path as image_path
            self.ids.uploaded_image.source = image_path  # changes the image in the editor.kv file to
the selected image
            current_directory = os.getcwd()  # retrieves the current working directory where the python
file is located
            new_filename = os.path.join(current_directory, 'uploaded_image.jpg')  # creates a new file
name
            # this new filename is inside the current directory titled uploaded_image
            os.rename(image_path, new_filename)  # it renames the uploaded image as the new file name
            # this saves the image into the same directory as the python file, so it can be used later
in the code
            print(f"Saved image as: {new_filename}")  # if all is well, a message is displayed in the
console

        self.dismiss_popup()  # once the selected file is saved, the popup is dismissed.

    def get_wing_dimensions(self):  # defines the function executed when the submit button is pressed
        try:
            global wing_width, wing_height  # allows the width and height to be accessed through the
entire script
            wing_width = int(self.wing_width_input.text)
            # gets the text input of the width and height and stores them as integers
            wing_height = int(self.wing_height_input.text)
            print(f"Wing width: {wing_width}, Wing height: {wing_height}")  # prints the dimensions
into the console
        except ValueError:  # if the inputs are not integer values, an error message is printed
            print("Invalid input. Please enter valid numeric values.")

class Editor(App):  # defines the editor class
    def build(self):  # builds the app and runs it
        return MyGrid()  # displays the app interface

Factory.register('MyGrid', cls=MyGrid)  # registers the two new widgets, so they can be instantiated in
the kv language
Factory.register('LoadDialog', cls=LoadDialog)

if __name__ == '__main__':  # ensures the app is only run when the python script is run directly
    Editor().run()  # runs the editor class
```

After we had successfully saved the image and dimensions, the next step was to output a dictionary containing a list of subpixel values for each RGB component of the pixel, ones representing that the subpixel was turned on, and zeros representing that the subpixel was turned off. This information could then be sent to the servo motors and the appropriate subpixels could be turned on, resulting in the uploaded image being displayed.

```
picture_dictionary = {}  # creates an empty dictionary
image = Image.open("uploaded_image.jpg").resize(
    (wing_width, wing_height))  # resizes the uploaded image to the dimensions of the user's wall
pixel = image.load()
width, height = image.size  # the width and height of the image are stored
count = 1  # sets count as 1, as we are starting with the first pixel
for x in range(0, width):
    for y in range(0, height):  # these two for loops allow us to access every pixel in the image
        rgb = pixel[x, y]  # gets the rgb values of the pixel at that specific point
        ratio = ()  # creates an empty tuple where the values of the subpixels will be stored
        for i in rgb:  # each of the rgb values of the pixel is divided by 3
            if i % 3 == 2:  # if the remainder is 2, the number is rounded up
                num = int(i / 3) + 1
            else:  # else, the number is rounded down
                num = int(i / 3)
            values = []  # creates an empty list where the values of the subpixels will be stored
            for j in range(0, num):  # this appends num number of ones to the list
                values.append(1)
            while len(values) < 85:  # while the length of the list is less than 85, a zero is added
                values.append(0)  # this results in the length of the values list equal to 85
            random.shuffle(values)  # the values are then shuffled to give us a randomly ordered list
            ratio = ratio + (values,)  # this adds the list of values into the RGB ratio
        name = "pixel" + str(count)  # sets the name of the ratio tuple as its pixel value
        picture_dictionary[name] = ratio  # adds the ratio tuple into the picture dictionary saved under its
name
        count += 1  # adds one to the count as we now move on to the next pixel
```

Thin Film Calculations

In order to calculate the thickness required for constructive interference to occur in each of the red, green, and blue subpixels, we used the equation $2t = \lambda_{n2}$, where twice the thickness of the film (the second ray travels in and then gets reflected back out) is equal to the wavelength of light we are after inside the thin film (since the path difference of the two waves must equal to the wavelength for constructive interference to occur), given by $\frac{\lambda}{n_2}$, where n_2 is the refractive index of our material, in this case silicon dioxide. [7]

	Wavelength (nm)	Refractive Index of Silicon Dioxide	Equation	Thickness of thin film (nm) (3sf)
Red	645	1.4634	$2t = \frac{645}{1.4635}$	220
Green	510	1.4678	$2t = \frac{510}{1.4678}$	174
Blue	440	1.4720	$2t = \frac{440}{1.4720}$	149

Materials

For our thin film, we decided to use silicon dioxide, as it has low absorption, allowing as much light as possible to be reflected or refracted. Its moderate refractive index ensures that there is a rough split between light getting reflected or refracted, resulting in maximum interference. It is also chemically stable and resistant, meaning it doesn't degrade easily over time, making it suitable for its long-lasting application in comparison to paint. The thickness of silicon dioxide can also be meticulously controlled using oxidation methods, crucial in achieving a thin film of an exact thickness to constructively interfere with a specific wavelength of light. Silicon dioxide is also abundantly available in the earth's crust and can be extracted sustainably from rice husks, an agricultural waste product. [8][9]

In order for as much light as possible to be reflected back into the thin film, we needed to use a highly reflective surface. It also needed to be denser than silicon dioxide (see the physics of light section), and as smooth as possible to prevent the light from scattering and therefore preventing it from interfering with the other ray. Therefore, we settled on using a silver-coated mirror as it has the highest reflectance percentage in the visible light spectrum out of metal-coated mirrors, ensuring 95% of light is reflected back. And since it is a metal-coated mirror, it meets our criteria of being denser than silicon dioxide. [10][11]

When our subpixels are in their "off" position, they are rotated to their sides which are covered in a dark surface capable of absorbing as much visible light as possible. The rest of the structure will also be covered in this material to ensure no other light is reflected off, as it could potentially be able to interfere with the light in the thin film. We decided to use black silicon, made by etching the surface of silicon to produce tiny pores which trap light through a process called hyperdoping. Its visible light absorbency is very near 100% and its thickness is only a few hundred nanometres, making it ideal for coating the sides of the thin film and the remaining structure. [12]

To protect the mechanism and ensure the user is still able to use the wall as normal (leaning objects against it, placing furniture), we needed to find a robust material, yet one with maximum transparency to prevent unwanted absorption of light. We decided to use polycarbonate, a transparent thermoplastic polymer which can transmit over 90% of light, almost as good as glass, yet is over 250 times as strong. Their durability and longevity also make them a more sustainable alternative to acrylic glass. Furthermore, they are 100% recyclable, and their diverse usage allows for various reusability. [13][14]

To stick all our components together and apply our structure to the wall, we looked for an environmentally friendly adhesive that would be strong enough to support our structure. We went for Eco Bond adhesives which have a strength of over 600 psi. They also have the lowest volatile organic compound VOCs concentration in the adhesive market, ensuring the safety of our users (find out more about VOCs in the Overview and Issue section on page 1). The adhesive is derived from renewable sources such as potatoes and corn. [15]

Design and Innovation

The Different Components

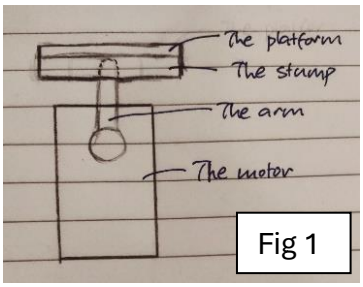
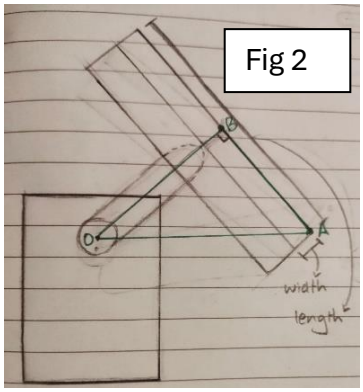


Fig 1

There were several different components that we needed to consider when designing our product. These included the motor, which turns the arm. The arm is then attached through a stump, which supports the platform on which light is refracted and interference occurs. Fig 1 shows our design for these components. Note that any reference to the arm length is taken from the pivot point to the tip.

The Platform

As already mentioned, the morpho butterfly has wings that consist of structures with multiple platforms called lamellae, making it look like a tree. However, for our purposes, we decided it would be more suitable to simply use one platform since it would be easier and less expensive to manufacture. It also works the same way as several platforms would, shown in thin film interference, which is explained in the Physics of Light section on page 3 . We also decided to make the length and height of the platform equal, as a square shape would ensure that each pixel is not elongated.



In the Thin Film Calculations section (page 7), we calculated the different widths of our platform depending on the colour of light it aims to reflect. Specifically, this was 220nm for red, 173nm for blue and 149nm for green. One of our considerations for the length of the platform is that each specific pixel of our product has a length of 10^6nm (1mm), which is equivalent to 18 “tree” structures. Therefore, each “tree” structure needs to lie in a square of length 10^6 / 18 = 55555nm. We decided that the length of the platform should be as close to this as possible in order to maximise the amount of light being reflected.

Another consideration for the length is that as the platform turns, it must not enter the space of the next “tree” structure, since then it might hit the platform in the next space. Drawing a line from O to any point on the platform, we see that OA would be the longest. Therefore, whilst turning, point A in Fig 2 would reach the furthest. Therefore, OA must be less than 55555/2 = 27777.5nm, since this is half the length of the space. We want to maximise BA to be as close to 55555/2 = 27777.5 nm as possible, since BA is half the length of the platform. OB must be large enough to leave room for the width of the platform, the width of the stump and the length of the arm. Furthermore, OA, AB and OB have to be Pythagorean triples since angle OBA needs to be a right angle. After experimenting with several different Pythagorean triples, we settled on OA = 25000nm, BA = 24000nm and OB = 7000nm. This makes the length of the platform 240000 x 2 = 48000nm. The height of the platform, which is the only dimension not depicted in Fig 1, will also take this value.

The Motor and The Arm

We decided to model the motor and the arm off of the servo motors that we had readily available at school. This is because they suited our purposes well, having the right shape and being able to turn our platform to the right angles. We were also planning to use them for our prototype. Now, we needed to work out the scale factor between the lengths of the motor and arm in our product, and what they would be in our prototype.

The length of OB, shown in Fig 2, must be 7000nm. If we suppose that this specific model is used to reflect red light, the width of the platform must be 220nm. Hence, the length of the arm, measured from the pivot point (O) to the tip, must be 7000 – 220= 6780nm. The length of the arm in our prototype is

exactly 2cm. Therefore, the scale factor between the prototype and our actual product must be 1cm:3390nm. We can find all the dimensions of the motor and the arm by applying this scale factor.

However, the width of the platform changes according to which colour it reflects. OB needs to stay the same length, so the length of the arm will change to accommodate this. The arm will be 6827nm long for blue, and 6851nm for green. Furthermore, the servo motor is not tall enough for the platform to fit beside it when it turns "off", so we decided to extend it by 21016nm to be 27797nm (8.2cm in the prototype) in total. The centre of gravity of the platform would actually also be slightly in front of the motor, due to the arm being extended out slightly in front of the motor. Hence, to prevent the structure from falling forwards, we decided to extend the front of the motor slightly by 2373nm (0.7cm in the prototype), 6780nm down from the top (in the prototype, this would be at the base of the motor).

The Stump

The stump is attached to the platform and has a hole through which the end of the arm sits. It is necessary to support the platform. We decided that the stump should extend the full length of the platform, but not the full height (which is the only dimension of the stump not depicted in Fig 3), since this extra mass would put too much burden on the arm and motor, as well as being more costly to manufacture.

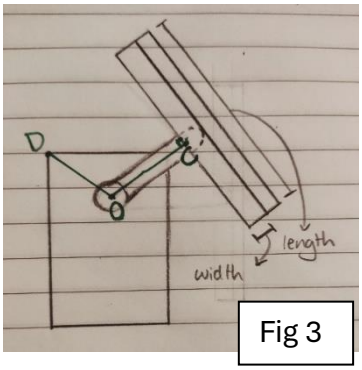


Fig 3

However, we needed to check if this design was valid by making sure that, when rotating, the stump did not crash into the motor. Referring to Fig 3, OC is the perpendicular length from the pivot point (O) to the bottom edge of the stump. Hence, it is the shortest distance from the pivot point to the stump. OD is the longest distance from the pivot point to an edge in the top half of the motor since D is the top left corner. Note that the distance between O and the right corner would be equal to OD by symmetry on the motor. We do not need to consider the bottom corners of the motor since the stump will not sweep past them.

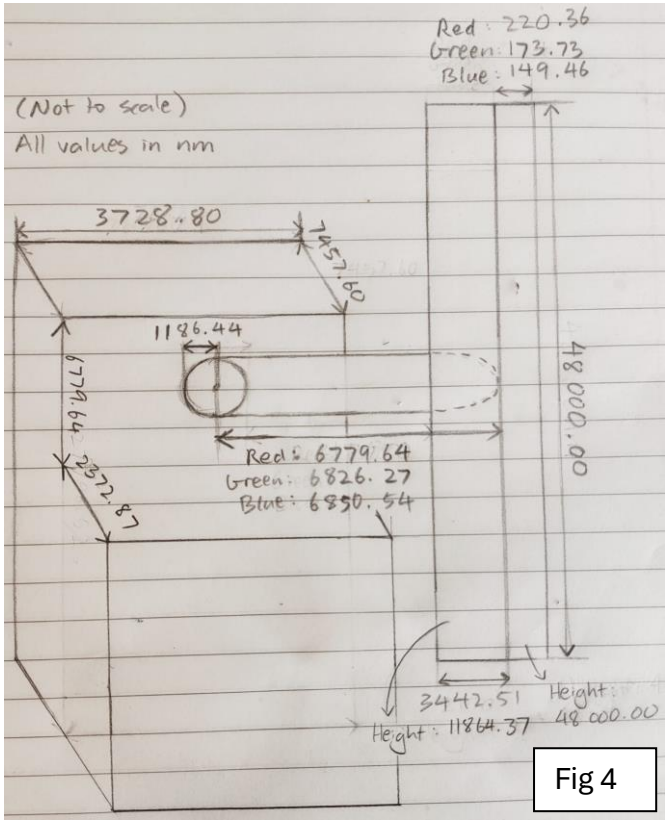


Fig 4

The length of OD is 2372.88nm, calculated from measuring the length from the pivot point to a top corner of the motor in our prototype and converting it using the scale factor. Suppose that the colour of light being reflected is red. The length of the whole arm would be 6779.64nm. Therefore, in order for OC to be larger than OD, the width of the stump just needs to be smaller than 6779.64 – 2337.13 = 4442.51. We decided to leave some more space between the stump and the corner of the motor to reduce the chance of accidents, reduce the cost of manufacture and accommodate for changes in the arm length according to what colour is being refracted. Hence, we decided to reduce the width of the stump by 1000nm to 3442.51nm.

It did not really matter what we made the height of stump, as long as its centre aligned with the centre of the platform. However, it did need to be large enough to support the platform, and small enough to reduce manufacture costs. We chose a height of 11864.37nm (3.5cm in our prototype).

Fig 4 shows a summary of the dimensions of our final product.

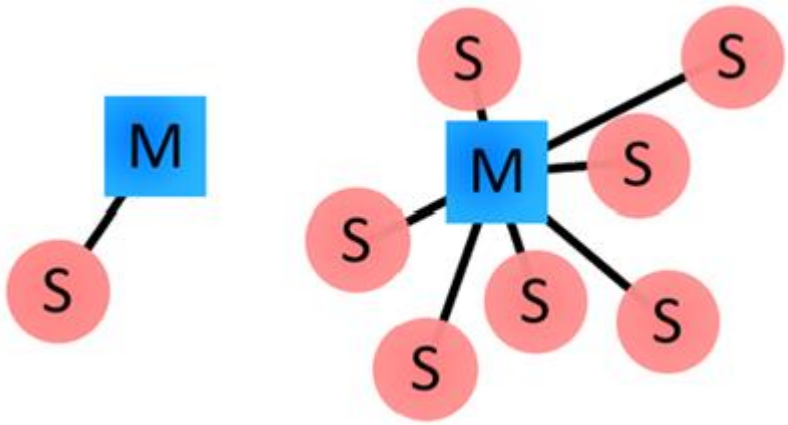
Bluetooth

In order to connect our app, where the user is able to upload their desired wallpaper image, to the motors on the wallpaper, we decided to utilise Bluetooth. See more about how the app works The App section on page 4.

Bluetooth is mainly designed for communicating over short distances of less than about 10 metres, which is suitable for our purposes, since the user can place the device with their app close to the wallpaper. It communicates through chips equipped with antennae that can receive or send information, and our motors will be built with these inside.

Bluetooth communicates though radio waves, which are sent out from a device in all directions. This poses the risk of data interference between different devices. The chances of this are minimised by sending information in bands of 79 different channels, each channel having a different frequency between 2.400 to 2.485 GHz. When two devices communicate, they pick one of the channels randomly. If that channel is taken, they randomly switch to another one, a technique known as spread-spectrum frequency hopping. Devices shift frequences thousands of times per second to reduce the risk of interference. Additionally, packages of data begin with a 72-bit long access code that synchronises communicating devices, making interference even less likely.

When two or more Bluetooth devices communicate, they form a network called a piconet. One device, known as the master and which in our case would be the user's device, acts as the overall controller of the network, whilst the others, known as the slaves and which would be the motors in our case, obey its instructions. However, only eight devices can be in a piconet at a time, meaning that the information from the app cannot be transmitted to all the motors at the same time. But the latest versions of Bluetooth, which are versions 5.0 and up, have very fast transmission rates, being up to 50Mbit/s, so this will not be an issue.



This diagram shows two piconets, with S representing a slave device, and M representing a master device. The piconet on the left shows a single-slave piconet whilst the one on the right shows a multi-slave piconet, with the maximum number of 8 devices.

We considered several alternatives to Bluetooth, including infrared radiation, but we found that, overall, Bluetooth had the most advantages. As discussed, it has many adaptive abilities that enable it to avoid interference and data misinterpretation, it can form a piconet allowing the communication of multiple devices at a time, and it also offers very fast communication. Meanwhile, technologies such as infrared have line of sight propagation, where the waves travel in a straight line, which is unfit for our purposes. [16][17][18]

The Prototype

We decided to put together a prototype to showcase the essentials of how our product works. In order to do this, we had to do a lot of research, experimenting, testing and improvement. Along the journey, we faced a multitude of mishaps, but emerged with a wealth of newfound skills.

The prototype consisted of an Arduino Uno as our microprocessor, a breadboard and 3 separate "tree" structures, each including a motor, an arm, a platform and a stump. Each separate structure would represent reflecting either red, green or blue light. Note that in our actual product, the Arduino, breadboard, and wires would not be necessary since the motors would be controlled with a Bluetooth chip.

Building the circuit

Since we are only using 3 motors, the 5V pin on the Arduino was able to supply enough power. However, we learnt this the hard way, having previously tried to use a 9V battery, which consequentially blew one of our motors. We attached the 5V pin to the positive bus on a breadboard, and the ground (GND) pin, which is used to provide a path for the electrical current to flow back to the power source, to the negative bus. We also attached our motors to the Arduino using pins 1, 2 and 3.

3D components

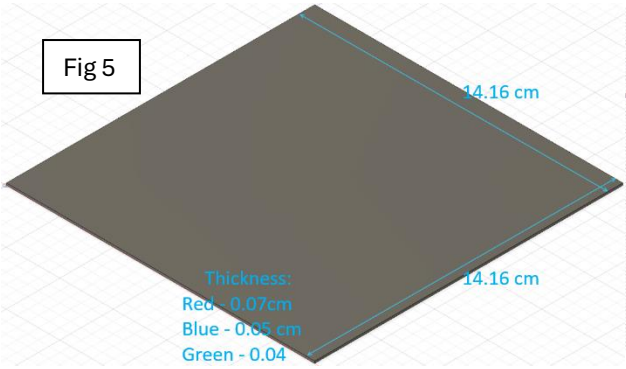


Fig 5

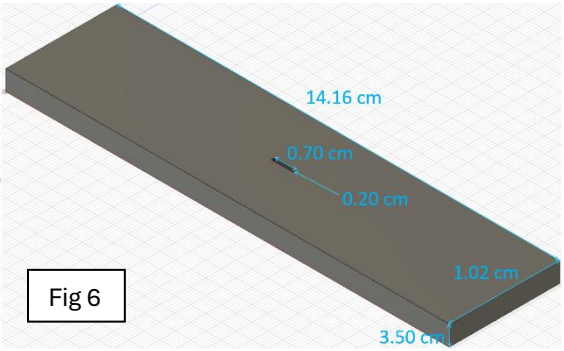


Fig 6

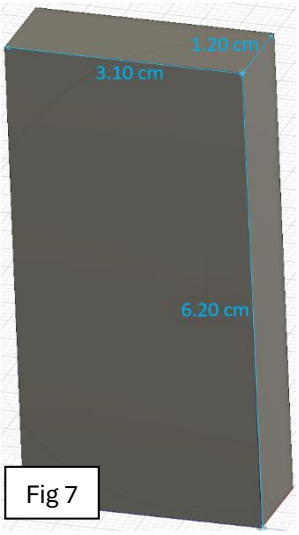
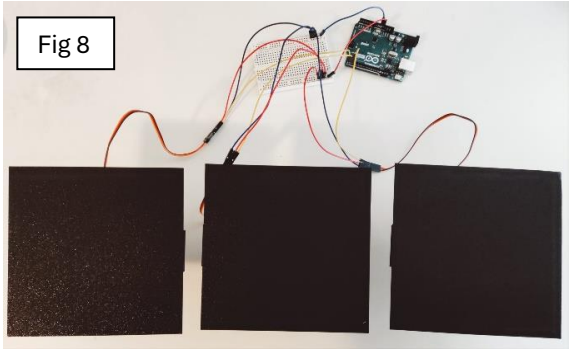


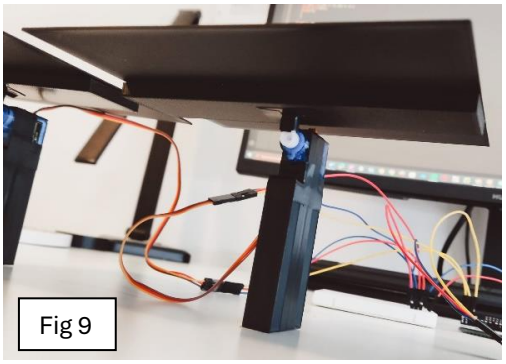
Fig 7

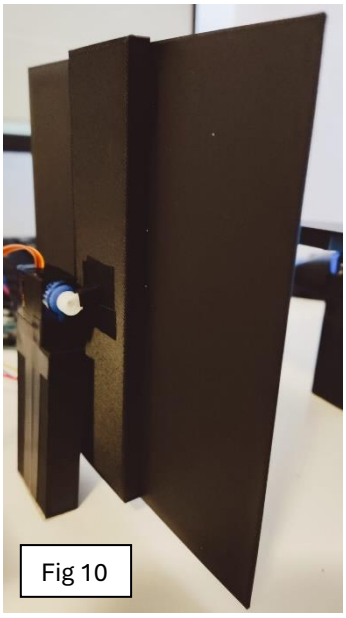
We designed our platform (Fig 5), stump (Fig 6) and motor extension (Fig 7) using the Fusion 360 software. The dimensions of each component are calculated according to those referred to in the Design and Innovation section on page 8 and scaled up using the scale factor 1cm:3389.82nm, the reasoning behind which is also explained in the Design and Innovation section. Additionally, each of the three platforms manufactured have a different width according to what colour it is reflecting.

With assistance from the CS department, these components were 3D printed. We then assembled them onto the circuit using all-purpose glue and electrical tape.

Fig 8 shows a bird's eye view of the final assembled prototype.


Fig 8


Fig 9


Fig 10

Coding

The code for the prototype operates such that if a value of 1 is sent to a specific motor, it will turn "on", which involves turning the platform so that it is as shown in Fig 9. If a 0 is sent, it will turn "off", which is shown in Fig 10. In our actual product, this process would be done through Bluetooth but for our prototype, it is done through sending this code to an Arduino. Hence, we wrote this code in the Arduino IDE, using C++, which was a new language that we learnt from scratch, a process that was challenging yet enjoyable.

```
1  #include <Servo.h>
2
3  Servo servo1;
4  Servo servo2;
5  Servo servo3;
6
7  int s1 = 1;
8  int s2 = 1;
9  int s3 = 1;
10 int pos = 0;
11
12 void setup() {
13   servo1.attach(1);
14   servo2.attach(2);
15   servo3.attach(3);
16
17   servo1.write(0);
18   servo2.write(0);
19   servo3.write(0);
20   delay(2000);
21
22   if (s1 == 1){
23     for (pos = 0; pos <= 90; pos += 1) {
24     servo1.write(pos);
25     delay(15);
26     }
27     }
28
29   if (s2 == 1){
30     for (pos = 0; pos <= 90; pos += 1) {
31     servo2.write(pos);
32     delay(15);
33     }
34     }
35
36   if (s3 == 1){
37     for (pos = 0; pos <= 90; pos += 1) {
38     servo3.write(pos);
39     delay(15);
40     }
41     }
42
43 }
44
45 void loop() {
46 }
```

Line 1 includes the servo library, allowing us to use functions from that library.

Lines 2-5 create 3 servo motors.

Lines 7-9 create 3 integer variables for each motor representing whether they are “on” or “off”, 1 representing “on” and 0 representing “off”. s1 corresponds servo1 and so on.

Line 10 creates an integer variable called pos, short for position, that will be used later to turn the motors.

Line 12 initiates the void setup, a function that runs only once at the beginning of the code.

Lines 13-15 attach each of the servo motors to each corresponding pin on the Arduino. servo1 is attached to pin 1, and so on.

Lines 17-19 reset all motors to degree 0, which is when the arm is pointed to the right and is “off”.

Line 20 contains a delay function, which specifies the amount of time the motor should take to turn to position 0. It has units of milliseconds, so the motor will take 2 seconds to get there.

Line 22 checks if the corresponding integer variable for servo1, which is s1, is 1, which tells the code that the servo 1 motor must be turned “on”.

Line 23 contains a for loop that is entered if s1 is indeed 1. To turn a servo motor, one specifies a certain degree in the .write() function. If a motor is “on”, it needs to turn to degree 90, where the arm is pointed upwards. This is done by increasing the pos variable by 1 every time the loop runs in line 23, and repeatedly writing it to the motor in line 24, causing it to turn 1 degree every time. The delay function in line 25 specifies that each degree should take 15 milliseconds (0.015 seconds) to turn to.

Lines 29-34 repeat the same if statement and for loop for servo2.

Lines 36-41 repeat the same if statement and for loop for servo3.

Lines 45-46 include a void loop, which is a loop that runs constantly as the program executes. We did not need to have anything in this loop, but it was necessary to include for the code.

Next Steps and Marketing

Our next steps would be to actually build our product. This would require the use of nanotech, which is defined as a branch of technology that deals with dimensions of 1-100 nanometres, known as the nanoscale. Our product would fall into this category since many dimensions include specifications up to 1 nanometre, and some components, such as those inside the motor, may fall within the nanoscale.

One of the main issues with nanotechnology is that at the nanoscale, statistical mechanical effects and quantum mechanical effects come into play. For example, mechanical, thermal and catalytic properties of materials are altered since their surface area to volume ratio is increased significantly. Additionally, once distances of 100 nm are reached, the quantum realm is reached. There are many reasons why this is important, one being that electrons are the primary thermal energy carriers in metals, and their distribution can be altered when reaching the nanoscale. However, researchers are also working on utilising some of these different properties to their advantage. [19][20]

A lot of research regarding the different ways to assemble nanostructures is also currently underway. The most popular methods at the moment are top-down approaches that involve creating smaller objects through using larger ones to dictate their assembly. For example, semiconductor device fabrication involves manufacturing semiconductor devices such as microcontrollers and memory chips using photolithography and physio-chemical processes, almost always involving the use of silicon [21]. This could be useful, for example, in manufacturing the Bluetooth chips for our product. However, top-down approaches are usually incredibly expensive due to the large number of technical steps required and are also limited due to technological instrument resolution capabilities. [22]

On the other hand, bottom-up approaches to nanofabrication are low-cost and seem more promising. They involve building larger devices from smaller ones. This includes, for example, self-assembly, where atoms or other nanoscale building blocks automatically organise themselves into ordered structures or patterns with some simple parameters [22]. Additionally, objects such as atomic force microscopes can deposit chemicals upon a surface in a desired pattern. This particular technique is a type of nanolithography. [19]

However, nanotech is a field that is quite complicated and still developing, as well as being incredibly expensive, so we would need further assistance from experts to assist us in building our product as well as sponsorships to help develop our idea further and make it into a fully functioning product. Collaboration with universities would also enable us to use their research labs and equipment to further our understanding and provide us with the resources necessary to make our product a reality. We would also be looking for sponsorship from environmental agencies keen to reduce both the negative impacts of paint on our health and the environment. We would also aim to market our product on, for example, billboards or trains, as well as working closely with property developers and technology companies to ensure that our product is optimised for consumers in the future.

Skills and Lessons Learnt

We learnt a variety of skills throughout this project, not just in computer science, but in maths, physics, chemistry, and practical skills as well. A huge challenge we faced was time management, as we underestimated the amount of time it would take to put our document together and finalise the various aspects of the prototype. Furthermore, thin film interference was a topic which was completely alien to us, and it took hours of committed research to understand the concept and then finally calculate thicknesses and dimensions. Debugging our codes was a cumbersome task, yet it helped us understand our code better and often improve it to run more efficiently, especially since we were processing huge dictionaries, containing thousands of lists to represent each pixel. Researching materials was an equally difficult task. They not only had to be sustainable, but also had to be suitable for our many elaborate purposes.

Putting the prototype together was also incredibly difficult, yet very rewarding. To begin with, through the planning and continuous altering of every detail of every component, we learnt to be flexible, observant and organised. This was especially true for our notes and diagrams, which we found challenging to keep legible for our future selves, resulting in considerable frustration later on. And, despite all of our forethought and scrutiny, we were inevitably met with many failures and forced to face many obstacles. However, through these hardships, we mastered skills of patience, problem-solving and the ability to learn from our mistakes instead of mourning over them. For example, when our circuit just wasn't working, we used a multimeter to test the wires, resulting in the discovery that one was broken. Previously, we were taught about multimeters in class, but now we knew how to use them as a real problem-solving tool.

Finally, we also gleaned invaluable collaboration and communication skills from this project. It could have not been done alone, and our teamwork has also brought us closer as people. We have developed a love for physics, maths, engineering, coding and physical computing together, a journey that has been incredibly enjoyable and rewarding.

External Support

As mentioned, our project was heavily inspired by a Physics Society talk about biomimicry, where we first encountered the fact that the morpho butterfly has no pigment in its wings, instead using interference to produce such a vivid blue colour. The Physics Department as a whole has been a huge support in helping us expand our knowledge of wave optics, especially in relation to thin film interference, and they have provided us with valuable resources to help strengthen the foundations of our project. The Computer Science and Creative Technology Department has helped us put our theories into practice, their constant support, guidance and availability of resources allowing us to design our prototype on Fusion, 3D print our components and use Kivy to build our app. We are incredibly lucky to have such brilliant departments at our disposal. Their unwavering support has opened doors to whole new aspects of both physics and technology, for which we are sincerely grateful.

Bibliography

1. Seladi-Schulman, J. (2019). *How Paint Fumes Affect Your Health and How to Prevent Exposure*. [online] Healthline. Available at: https://www.healthline.com/health/paint-fumes#about-paint.
2. www.greenspec.co.uk. (n.d.). *Paint: Health and the Environment*. [online] Available at: https://www.greenspec.co.uk/building-design/paint/.
3. PollutionIssues (2008). *How Paint Pollution Effects the Environment*. [online] Pollution Issues. Available at: https://pollutionissues.co.uk/how-paint-pollution-effects-environment/.
4. openstax.org. (n.d.). *3.4 Interference in Thin Films - University Physics Volume 3 | OpenStax*. [online] Available at: https://openstax.org/books/university-physics-volume-3/pages/3-4-interference-in-thin-films.
5. Khan Academy. (n.d.). *Thin Film Interference part 1 (video)*. [online] Available at: https://www.khanacademy.org/science/physics/light-waves/interference-of-light-waves/v/thin-film-interference-part-1.
6. GeeksforGeeks. (2020). *Kivy Tutorial*. [online] Available at: https://www.geeksforgeeks.org/kivy-tutorial/.
7. refractiveindex.info. (n.d.). *Refractive index of SiO2 (Silicon dioxide, Silica, Quartz) - Rodriguez-de_Marcos*. [online] Available at: https://refractiveindex.info/?shelf=main&book=SiO2&page=Rodriguez-de_Marcos.
8. SCIENCE OF THIN FILMS Deposition Overview for Microsystems 'Rainbow Wafer' [Courtesy of MJ Willis, personal collection.]. (n.d.). Available at: https://nanohub.org/resources/26367/download/Presentation-ScienceOfThinFilmsAC.pdf.

9. De Gruyter Conversations. (2019). *Going the green way: Rice husks as an eco-friendly source of silica*. [online] Available at: https://blog.degruyter.com/going-green-way-rice-husks-eco-friendly-source-silica/.
10. R.I.N.T. (n.d.). *Aluminum Coated Mirrors: Do You Really Need It? This Will Help You Decide!* [online] www.clzoptics.com. Available at: https://www.clzoptics.com/news/aluminum-coated-mirrors-this-will-help-you-decide.html.
11. JDSU (n.d.). *Mirrors: Coating Choice Makes a Difference*. [online] www.photonics.com. Available at: https://www.photonics.com/Articles/Mirrors_Coating_Choice_Makes_a_Difference/a25501.
12. Liu, X., Coxon, P.R., Peters, M., Hoex, B., Cole, J.M. and Fray, D.J. (2014). Black silicon: fabrication methods, properties and solar energy applications. *Energy Environ. Sci.*, 7(10), pp.3223–3263. doi:https://doi.org/10.1039/c4ee01152j.
13. Omnexus (2018). *Polycarbonate (PC) Plastic: Properties, Uses, & Structure - Guide*. [online] Specialchem.com. Available at: https://omnexus.specialchem.com/selection-guide/polycarbonate-pc-plastic.
14. Kate_Acme, A. (2023). *Recycling Polycarbonate: How, Why & Where*. [online] Acme Plastics. Available at: https://www.acmeplastics.com/content/recycling-polycarbonate-how-why-where/.
15. ecobondadhesives.com. (n.d.). *Eco-Bond - The Environmentally Safe Adhesive & Sealant*. [online] Available at: https://ecobondadhesives.com/performance.html
16. Explain that Stuff. (2009). *How does Bluetooth work?* [online] Available at: https://www.explainthatstuff.com/howbluetoothworks.html#howwork.
17. Franklin, C. and Pollette, C. (2000). *How Bluetooth Works*. [online] HowStuffWorks. Available at: https://electronics.howstuffworks.com/bluetooth.htm.
18. Davies, M. (2021). *Infrared Communication: Advantages and Disadvantages*. [online] Konsyse. Available at: https://www.konsyse.com/articles/infrared-communication-advantages-and-disadvantages/#:~:text=Below%20are%20the%20drawbacks%20and%20limitations%20of%20IR.
19. Wikipedia. (2024). *Nanotechnology*. [online] Available at: https://en.wikipedia.org/wiki/Nanotechnology#Larger_to_smaller:_a_materials_perspective
20. Unique Properties. (2016). *Essentials in Nanoscience and Nanotechnology*, pp.326–360. doi:https://doi.org/10.1002/9781119096122.ch8.
21. Wikipedia. (2020). *Semiconductor device fabrication*. [online] Available at: https://en.wikipedia.org/wiki/Semiconductor_device_fabrication.
22. Berbezier, I. and M. De Crescenzi (2015). Self-assembly of nanostructures and nanomaterials. 6, pp.1397–1398. doi:https://doi.org/10.3762/bjnano.6.144.
23. MyBib Contributors (2019). *Harvard Referencing Generator – FREE – (updated for 2019)*. [online] MyBib. Available at: https://www.mybib.com/tools/harvard-referencing-generator.



Document written by Silvia Muresan and Zhixuan (Sharon) Wang